

Machine Learning

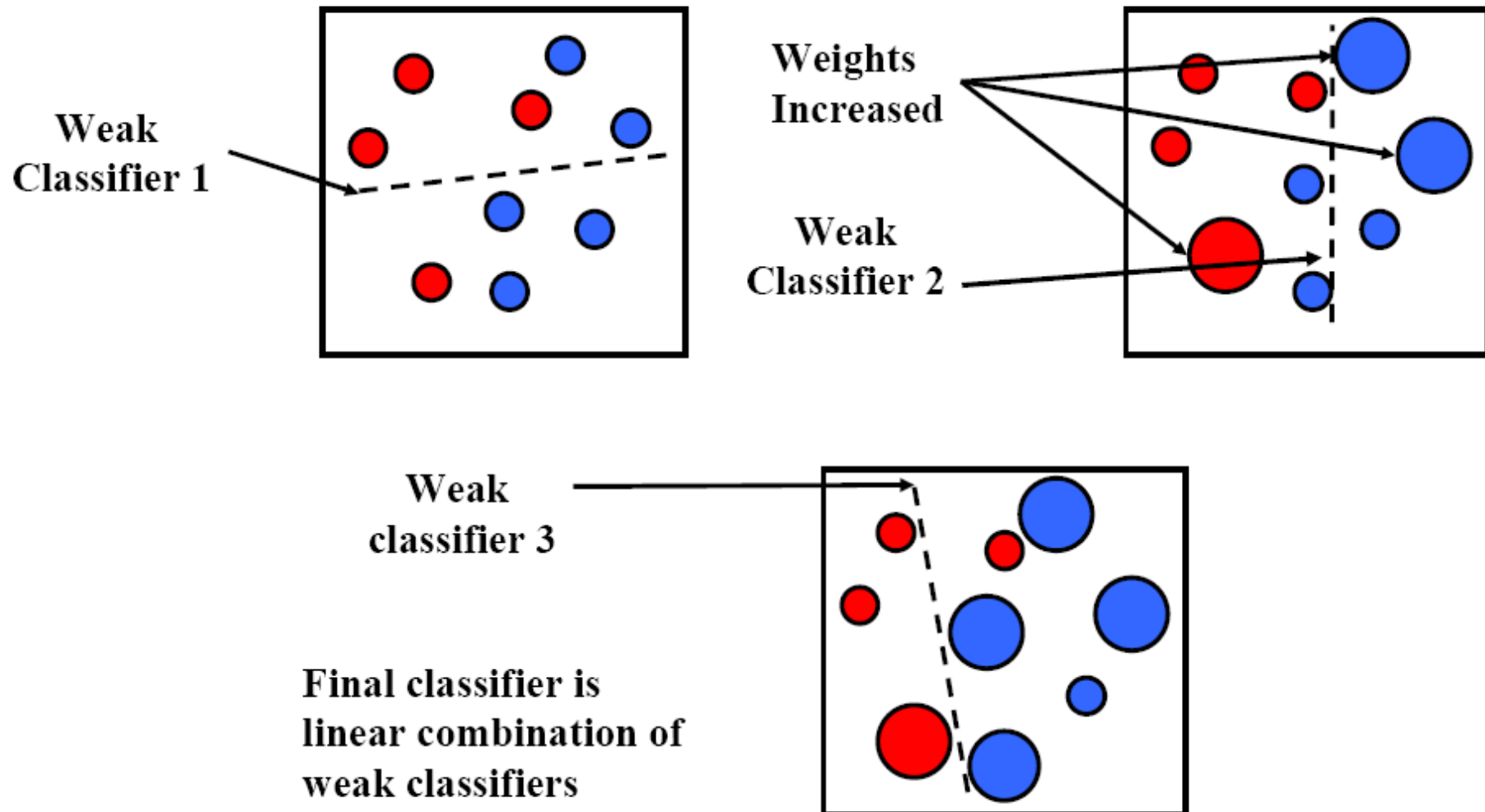
Prof. Dr.-Ing. Bodo Rosenhahn

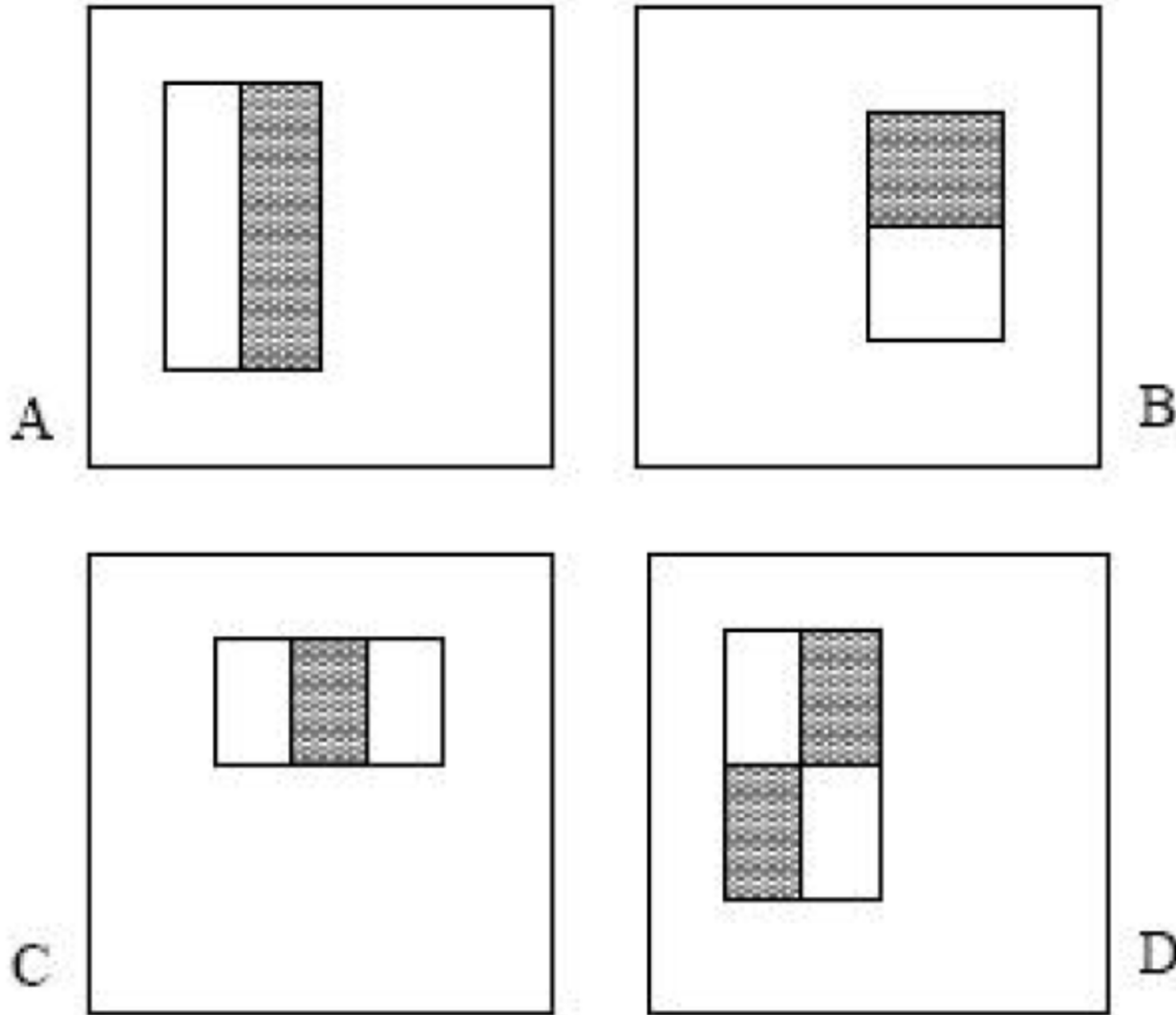
Institut für Informationsverarbeitung



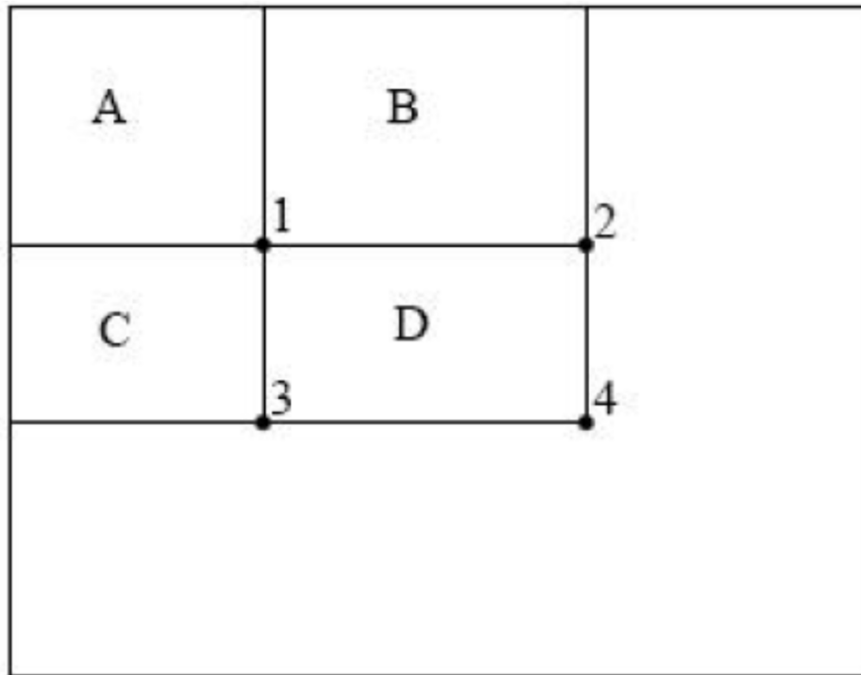
Themen

Termin	Inhalt
15. April 2026	Einleitung, Clustering
22. April 2026	Dimensionsreduktion (PCA, MDS), Anomalie-Detektion
29. April 2026	Grundlagen Statistik, Bayes Classifier
06. Mai 2026	Support Vector Machines
13. Mai 2026	Random Forest, Adaboost
20. Mai 2026	Hidden Markov Models
27. Mai 2026	Vorlesungsfrei (Exkursionswoche)
03. Juni 2026	Neuronale Netze I (Linear Layer, Autodiff)
10. Juni 2026	Neuronale Netze II (Faltungsnetze)
17. Juni 2026	Architekturen I (Autoencoder, GAN, LSTM)
24. Juni 2026	Architekturen II (Explainability, Uncertainty, Transformer)
01. Juli 2026	Architekturen III (SSM, GNN)
08. Juli 2026	Spieltheorie, MCTS, Reinforcement Learning
15. Juli 2026	Bufferslot





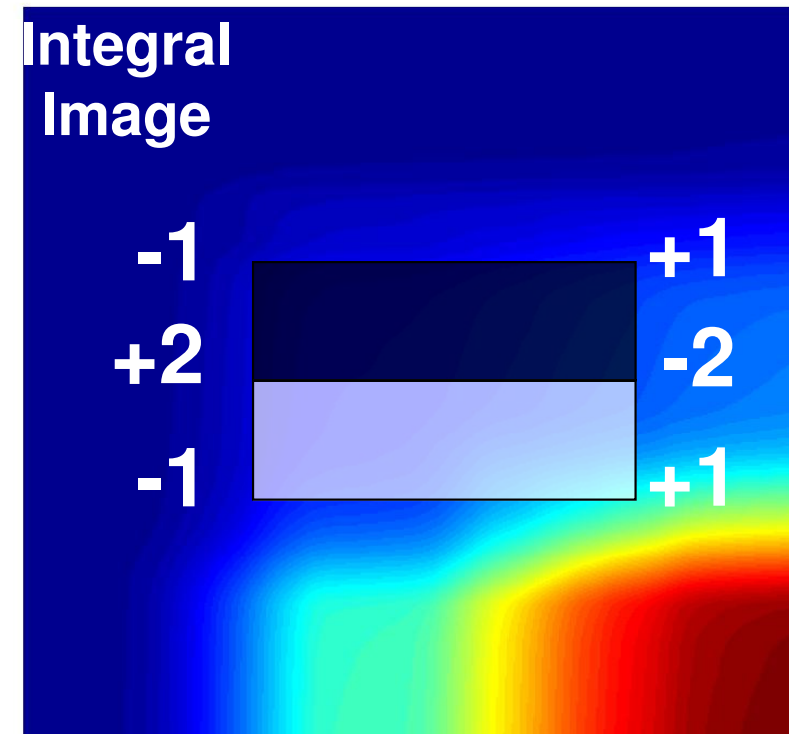
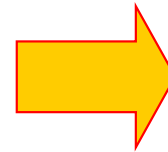
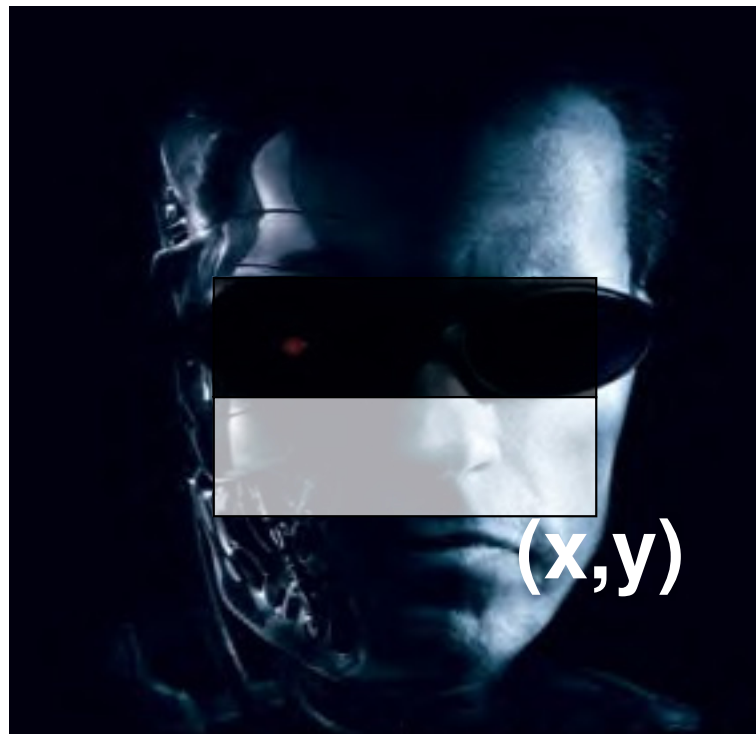
- Rechtecke robuster als Pixel
- Einfach zu skalieren
- Einfacher Vergleich der dunklen und hellen benachbarten Flächen



- Schnelle Berechnung der Rechtecke durch *Integralbild*

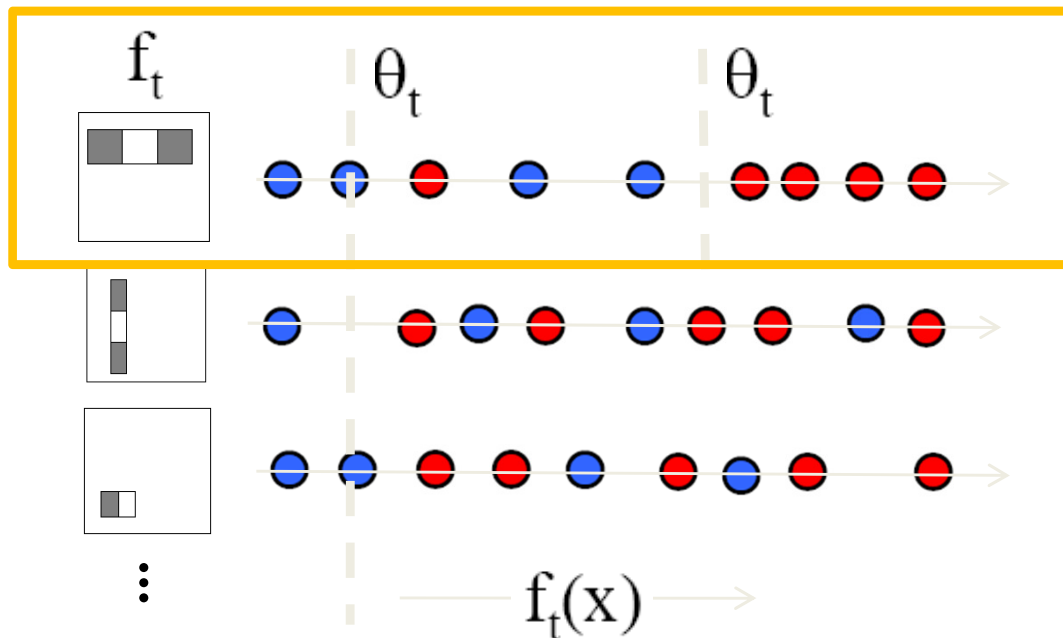
4 Berechnungen pro Rechteck

$$\begin{aligned} D &= 1 + 4 - (2 + 3) \\ &= A + (A + B + C + D) - (A + C + A + B) \\ &= D \end{aligned}$$



- Konstanter Zeitaufwand z.B. bei zwei gleichen Rechtecken nur noch 6 Rechenschritte

- Wähle ein Rechteck-Feature + Schwellwertbildung um am besten positiven und negative Beispiele zu trennen



Ergebnis eines möglichen
Features für Gesichter
und Nicht-Gesichter

Ergebnis: schwacher Classifier:


$$h_t(x) = \begin{cases} +1 & \text{if } f_t(x) > \theta_t \\ -1 & \text{otherwise} \end{cases}$$

Für die nächste Runde regewichte
die Beispiele abhängig der Fehler
und wähle eine andere
Filter/Schwellwert-Kombination

Slide credit: Kristen Grauman

(Tafel)

[Viola & Jones, CVPR 2001]

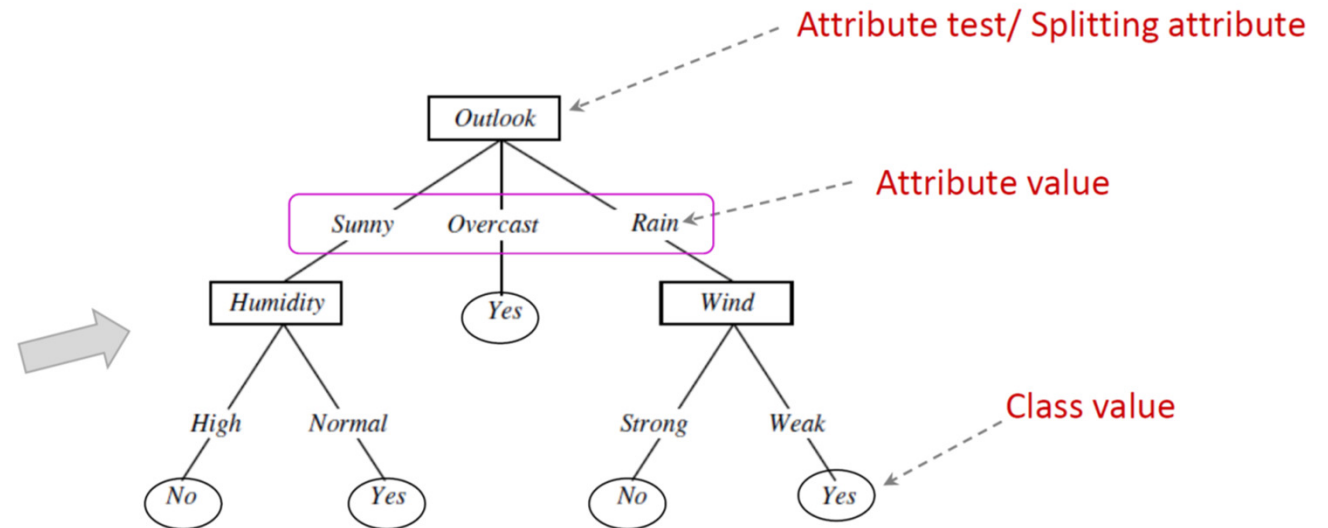
- Jeder Baum wird zufällig erstellt
 - Wähle zufällig Trainingsobjekte
 - Wähle zufällig Features
- Bäume werden binär aufgebaut, jeder Knoten wird in maximal 2 Kinder unterteilt
- Wähle $\sqrt{\text{Anzahl aller Features}}$ zum Aufbau eines Baumes
- Auch einzelne Bäume können sehr robust sein
- Einzelner Baum entspricht Nearest-Neighbor Klassifikator

Slide credit: Florian Baumann

Entscheidungsbäume

Training set

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



size: small medium large

colour: red blue green

shape: brick wedge sphere pillar

%% yes

medium blue brick

small red sphere

large green pillar

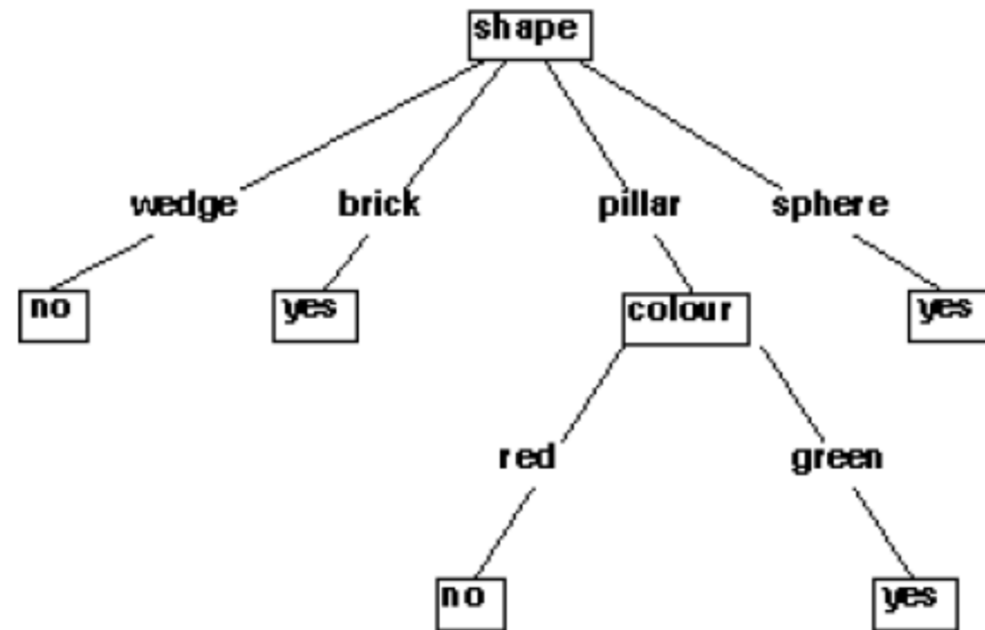
large green sphere

%% no

small red wedge

large red wedge

large red pillar



7 Instanzen (Beispiele), 3 Features (größe, Farbe, shape)

Slide credit: Florian Baumann

<http://www.cse.unsw.edu.au/~billw/cs9414/notes/ml/06prop/id3/id3.html>

Der anfängliche Entscheidungsbaum ist ein Knoten mit allen Beispielen.

Es gibt 4 positive Beispiele und 3 negative Beispiele

d. h. die Wahrscheinlichkeit eines positiven Ergebnisses beträgt $4/7 = 0,57$;

Die Wahrscheinlichkeit für negativ beträgt $3/7 = 0,43$

Die Entropie beträgt: $-(0,57 * \log 0,57) - (0,43 * \log 0,43) = 0,99$

Bewertung möglicher Aufteilungen:

1. Teste Aufteilung nach Größe, die drei Werte hat: *groß*, *mittel* und *klein*.

Es gibt vier Instanzen mit Größe = large.

Es gibt zwei große positive Beispiele und zwei große negative Beispiele.

Die positive Wahrscheinlichkeit beträgt 0,5

Die Entropie beträgt: $-(0,5 * \log 0,5) - (0,5 * \log 0,5) = 1$

Es gibt ein kleines positives und ein kleines negatives Beispiel

Die Entropie beträgt: $-(0,5 * \log 0,5) - (0,5 * \log 0,5) = 1$

Es gibt es nur ein mittleres Positives und kein mittleres Negatives, daher ist die Entropie 0.

Die erwarteten Informationen für eine Aufteilung nach Größe sind:

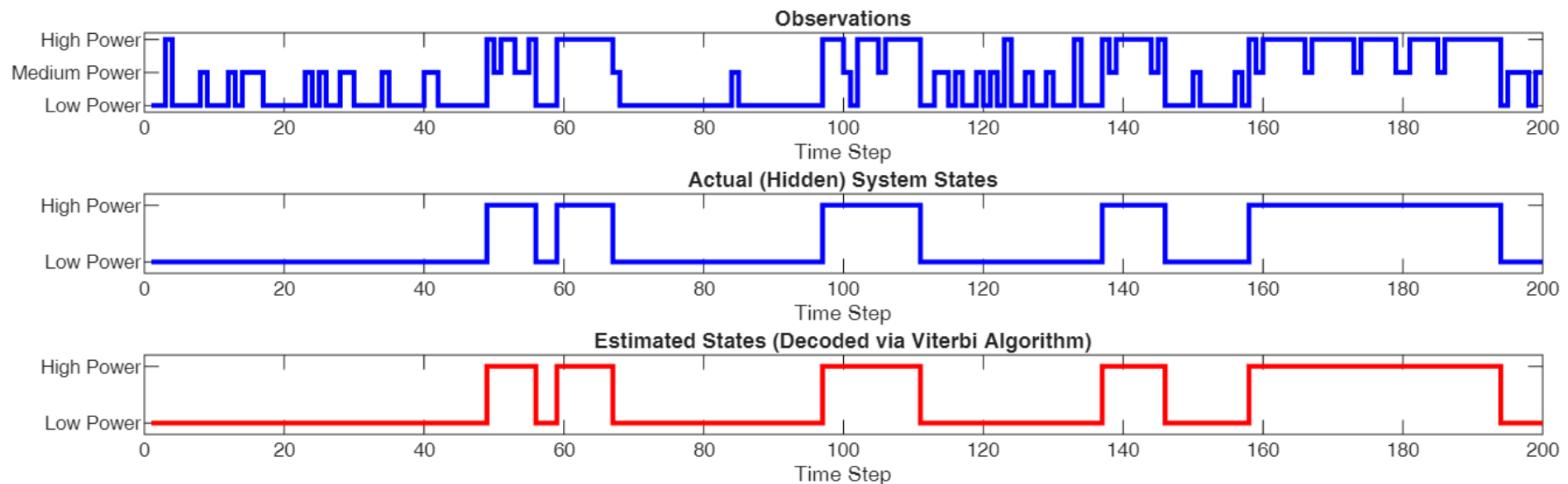
$$\left(1 \times \frac{4}{7}\right) + \left(1 \times \frac{2}{7}\right) + \left(0 \times \frac{1}{7}\right) = 0.86$$

Slide credit: Florian Baumann

<http://www.cse.unsw.edu.au/~billw/cs9414/notes/ml/06prop/id3/id3.html>

- Trends: Kurzzeitveränderungen vs. Langzeitentwicklungen
- Saisonalität: Wiederholende zyklische Muster
- Anwendungen: Preisvorhersage, Logistik, Sprach/Audioanalyse
- HMMs : Ein statistisches Modell, das zur Beschreibung der Entwicklung beobachtbarer Ereignisse verwendet werden kann, die von internen Faktoren abhängen, die nicht direkt beobachtbar sind.

- Beispiel: Regelung eines Atomkraftwerks, basierend auf verrauschten Beobachtungen



<https://exploreuclear.com/nuclear-chain-reaction-simulation>

Beispiel

- Ein Textgenerator hat ein Lexikon mit drei Wörtern
- von denen an jeder Position jedes auftreten kann
($\Omega = \{geschickt, werden, wir\}$)
- wir beobachten an jeder Position, welches Wort generiert wurde
- Sei
 - X_1 das Wort zum ersten Beobachtungszeitpunkt
 - X_2 das Wort zum zweiten Beobachtungszeitpunkt, usw.
- Dann ist die Folge der Wörter ein stochastischer Prozess mit diskreter Zufallsvariable und diskretem Zeitparameter

- Für die vollständige Beschreibung eines Zufallsprozesses mit diskretem Zeitparameter benötigt man

1. die Anfangswahrscheinlichkeit:

die für jeden Zustand angibt, mit welcher Wahrscheinlichkeit er als Zustand X_1 beobachtet werden kann (d.h. den Startzustand bildet)

$$\pi_i = P(X_1 = s_i)$$

2. die Übergangswahrscheinlichkeit:

die für jeden Zustand angibt, mit welcher Wahrscheinlichkeit er in einer Zustandsfolge auftritt:

$$P(X_{t+1} = x_{t+1} \mid X_1 = x_1, X_2 = x_2, \dots, X_t = x_t)$$

Definition 3

Eine **Markow-Kette** ist ein spezieller stochastischer Prozess,
bei dem zu jedem Zeitpunkt
die Wahrscheinlichkeiten aller zukünftigen Zustände
nur vom momentanen Zustand abhängen
(= **Markow-Eigenschaft**)

d.h. es gilt:

$$P(X_{t+1} = x_{t+1} \mid X_1 = x_1, X_2 = x_2, \dots, X_t = x_t) = \\ P(X_{t+1} = x_{t+1} \mid X_t = x_t)$$

Brants, 1999: 30

- Stochastische Übergangsmatrix A

$$a_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$$

$$\forall i, j \quad a_{ij} \geq 0$$

$$\forall i \quad \sum_{j=1}^N a_{i,j} = 1$$

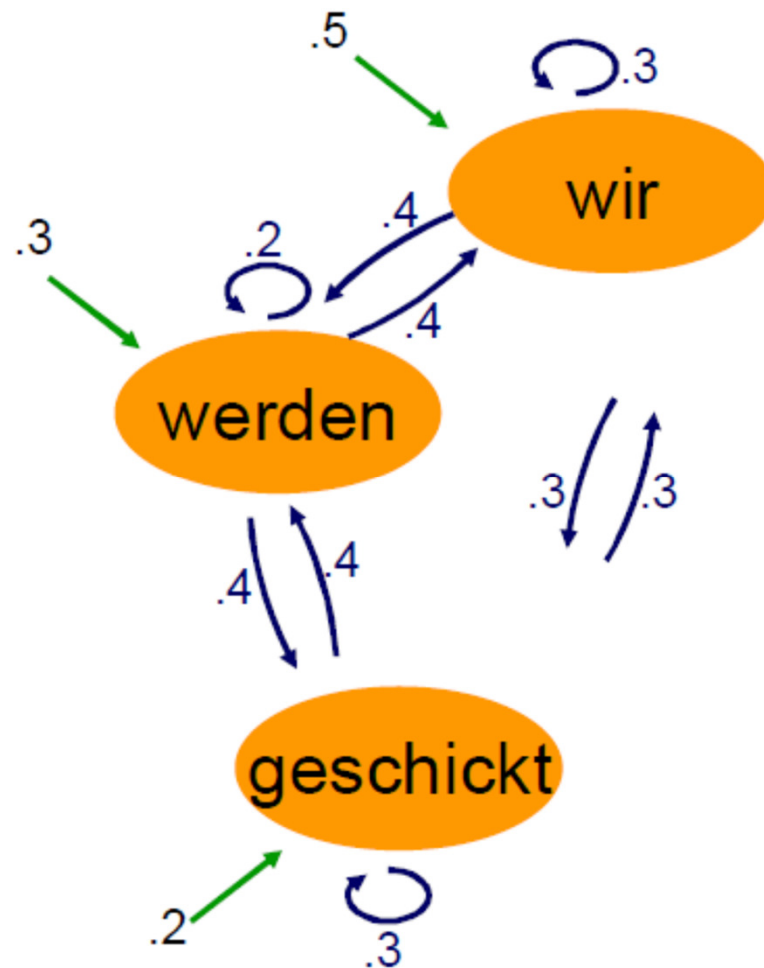
$X_t = s_i$	$X_{t+1} = s_j$		
	geschickt	werden	wir
geschickt	.3	.4	.3
werden	.4	.2	.4
wir	.3	.4	.3

- Anfangswahrscheinlichkeiten Π

$$\pi_i = P(X_1 = s_i)$$

$$\sum_{i=1}^N \pi_i = 1$$

X_t	π
geschickt	.2
werden	.3
wir	.5



$$P(X_1, \dots, X_T)$$

$$= P(X_1)P(X_2 \mid X_1)P(X_3 \mid X_2, X_1) \dots P(X_T \mid X_1, \dots, X_{T-1})$$

für eine Markow-Kette gilt:

$$= P(X_1)P(X_2 \mid X_1)P(X_3 \mid X_2) \dots P(X_T \mid X_{T-1})$$

$$= \pi_{X_1} \prod_{t=1}^{T-1} a_{X_t X_{t+1}}$$

$$P(X_1 = \text{wir}, X_2 = \text{werden}, X_3 = \text{geschickt})$$

$$= P(X_1 = \text{wir}) \cdot$$

$$P(X_2 = \text{werden} \mid X_1 = \text{wir}) \cdot$$

$$P(X_3 = \text{geschickt} \mid X_2 = \text{werden})$$

$$= (.5 \times .4 \times .4) = 0.08$$

- Ein Markow-Modell ordnet jedem Zustand (andere Variante: jedem Zustandsübergang) eine **Ausgabe** zu, die ausschließlich vom aktuellen Zustand (bzw. Zustandsübergang) abhängig ist
- Ausgabe: Sequenz von Ereignissen, die die Beobachtungen in der Beobachtungssequenz repräsentieren

Zur Unterscheidung auch **Visible Markov Model (VMM)** genannt

Das **Hidden Markov Model** (*HMM*), benannt nach dem russischen Mathematiker Andrei Andrejewitsch Markow, ist ein stochastisches Modell, das sich durch zwei Zufallsprozesse beschreiben lässt.

Der erste Zufallsprozess entspricht dabei einer Markow-Kette, die durch Zustände und Übergangswahrscheinlichkeiten gekennzeichnet ist. Die Zustände der Kette sind von außen jedoch nicht direkt sichtbar (sie sind **hidden**, verborgen). Stattdessen erzeugt ein zweiter Zufallsprozess zu jedem Zeitpunkt beobachtbare Ausgangssymbole gemäß einer zustandsabhängigen Wahrscheinlichkeitsverteilung. Die Aufgabe besteht häufig darin, aus der Sequenz der Ausgabesymbole auf die Sequenz der verborgenen Zustände zu schließen.

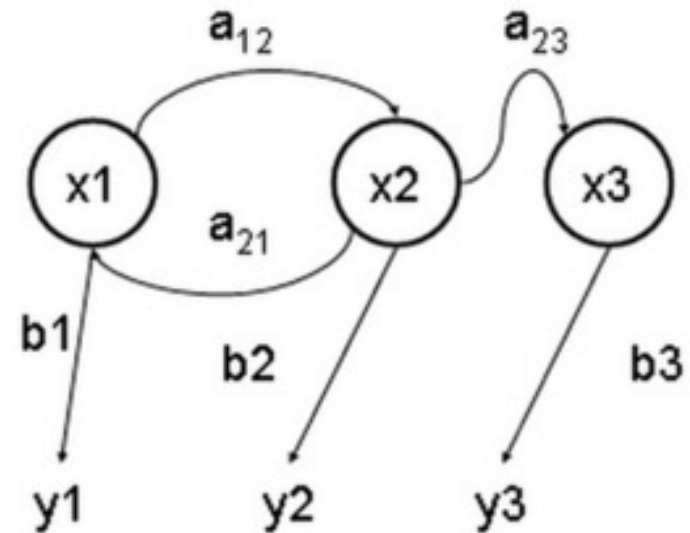
http://de.wikipedia.org/wiki/Hidden_Markov_Model

Formal definiert man ein HMM als Quintupel $\lambda = (S, A, B, \pi, V)$ mit

- $S = \{s_1, \dots, s_n\}$ – Menge aller Zustände
- $A = \{a_{ij}\}$ – Zustandsübergangsmatrix, wobei a_{ij} die Wahrscheinlichkeit angibt, dass nach Zustand s_i in Zustand s_j gewechselt wird.
- $B = \{b_1, \dots, b_n\}$ – Menge der Emissionswahrscheinlichkeitsverteilungen bzw. Dichten
 - $b_i(x)$ – Wahrscheinlichkeit, im Zustand s_i die Beobachtung x zu machen
 - π – Anfangswahrscheinlichkeitsverteilung mit $\pi(i)$ Wahrscheinlichkeit, dass s_i der Startzustand ist
 - V – Merkmalsraum (oft als Ausgabealphabet bezeichnet), also der Definitionsbereich von b_i . Dieser kann diskret oder kontinuierlich sein - davon hängt ab, ob b_i eine Wahrscheinlichkeit oder eine Wahrscheinlichkeitsdichte ist.

Intuitiv: Automat mit Wahrscheinlichkeitsdichten + Emission einer Beobachtung

x – (verborgene) Zustände des Markow-Modells
a – Übergangswahrscheinlichkeiten
b – Emissionswahrscheinlichkeiten
y – (sichtbare) Ausgabesymbole



Die Gemütszustände eines Professors:

$$h_t \in S = \{ \text{Schlecht } \text{☹} , \text{neutral } \text{☺} , \text{gut } \text{😊} \}$$

Je nachdem wie der Prof. gelaunt ist, verändert sich die WK für eine Note (1..3)

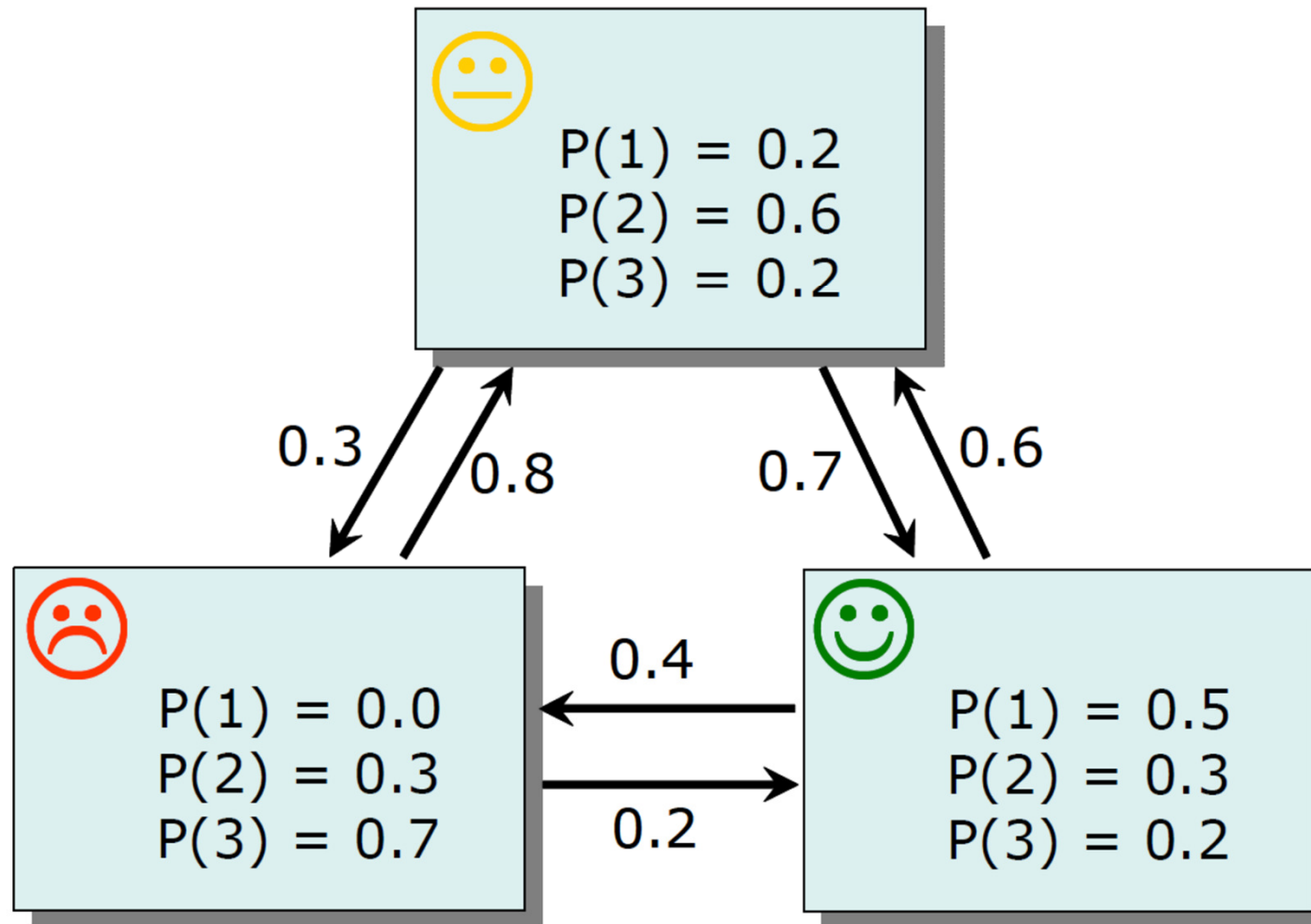
Noten Gemüt	1	2	3
☹	0.0	0.3	0.7
☺	0.2	0.6	0.2
😊	0.5	0.3	0.2

Zwischen den Gemütszuständen gibt es die folgenden ÜbergangswK's:

Q:

			
	0	0.8	0.2
	0.3	0	0.7
	0.4	0.6	0

Die Gemütszustände sind nicht ablesbar (hidden), die Noten die Beobachtung.
Das Gemüt sei nur abhängig vom Vortag (Markov-eigenschaft)



Problem 1: Wie wahrscheinlich ist eine Beobachtungssequenz bei gegebenem HMM?

Problem 2: Wie wahrscheinlich ist ein Zustand s zum Zeitpunkt i bei einer Beobachtung?

Problem 3: Wie ist die optimale Hintergrundsequenz bei gegebener Beobachtungssequenz

Problem 4: Wie optimiert man die Parameter, also die Übergangswahrscheinlichkeiten und Ausgabewahrscheinlichkeiten eines HMMs?

Evaluierungsproblem (1)

Gegeben ist ein HMM λ sowie eine Beobachtung O . Gesucht ist die Wahrscheinlichkeit $P(O | \lambda)$ (auch Baum-Welch-Score genannt), d. h. die Wahrscheinlichkeit dass die Beobachtung O unter gegebenem HMM λ gemacht wird.

Analysiere alle Zustandsfolgen die O produzieren können:

$$P(O|\lambda) = \sum_{X \in \mathcal{Z}} P(O, X|\lambda) = \sum_{X \in \mathcal{Z}} P(O|X, \lambda)P(X|\lambda).$$

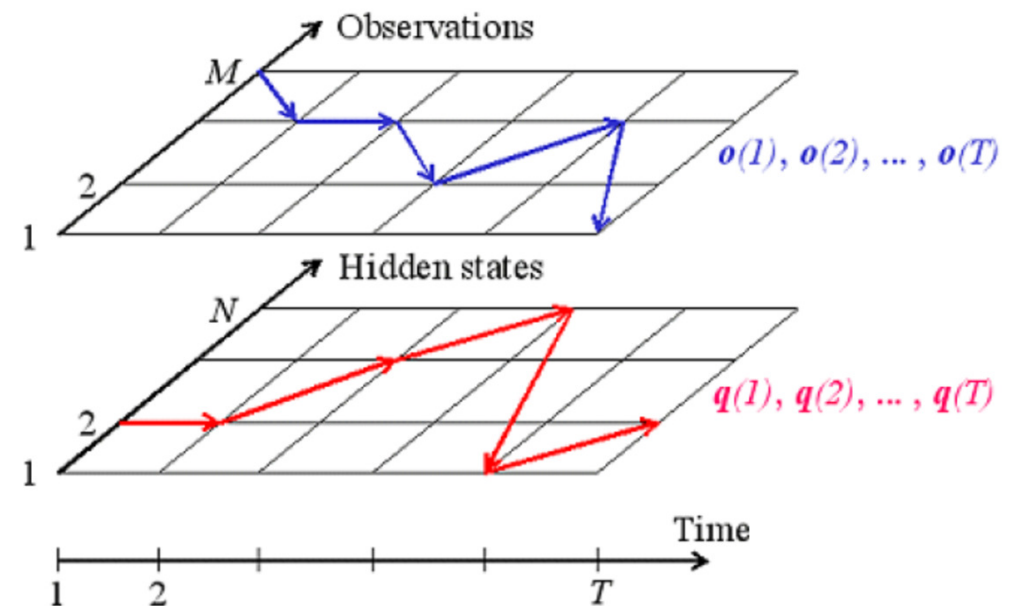
Bei der naiven Berechnung benötigt man exponentiell viele Operationen.

Effizientes Verfahren: Likelihood-rekursion

Verwendet das Prinzip der dynamischen Programmierung

Die Wahrscheinlichkeit zum Zeitpunkt t bei gegebener Beobachtung $O = (o_1, o_2, \dots, o_t)$ im Zustand i zu sein, ist:

$$\alpha_t(i) = P(o(1), o(2), \dots, o(t) \mid q(t) = q_i).$$



<http://de.wikipedia.org/wiki/Forward-Algorithmus>

1. Initialisierung

$$\alpha_1(i) = p_i b_i(o(1)) , i = 1, \dots, N$$

EingangsWK

EmissionsWK

2. Rekursion,

$$\alpha_{t+1}(i) = \left[\sum_{j=1}^N \alpha_t(j) a_{ji} \right] b_i(o(t+1))$$

Vorherige WK

ÜbergangsWK

EmissionsWK

3. Terminierung

$$P(o(1)o(2)\dots o(T)) = \sum_{j=1}^N \alpha_T(j)$$

Beispiel : „Der gut gelaunte Professor“

Noten Gemüt	1	2	3
	0.0	0.3	0.7
	0.2	0.6	0.2
	0.5	0.3	0.2

			
	0	0.8	0.2
	0.3	0	0.7
	0.4	0.6	0

Frage: wie gross ist die WK dass die Notenkette {2,3,1} vergeben wird ?

Startwahrscheinlichkeiten

$$\pi_0(\text{☹}) = 0.2$$

$$\pi_0(\text{😊}) = 0.3$$

$$\pi_0(\text{😄}) = 0.5$$

$$p_{1rs} = \pi_0(s) * P_s(d_1|\theta)$$

$$p_{1r\text{☹}} = 0.2 * 0.3 = 0.06$$

$$p_{1r\text{😊}} = 0.3 * 0.6 = 0.18$$

$$p_{1r\text{😄}} = 0.5 * 0.3 = 0.15$$

Initialisierung

In diesem Beispiel ist p_{1rs} die W-keit, dass ich zu Beginn der Untersuchung ($t=1$) den Gemütszustand s mit der gegebenen W-keit $\pi_0(s)$ habe * der W-keit das die Note 2 gegeben wird.

Frage: wie groß ist die WK dass die Notenkette {2,3,1} vergeben wird ?

Forward Induktion (über $t=2, \dots, T-1$): $p_{trs} = \sum_r p_{t-1rs} * q(r,s) * P_s(d_t|\theta)$

t=2:

$$p_{2r_{\otimes}} = [(0.06*0) + (0.18*0.3) + (0.15*0.4)] * 0.7 = \underline{0.0798}$$

$$p_{2r_{\odot}} = [(0.06*0.8) + (0.18*0) + (0.15*0.6)] * 0.2 = \underline{0.0276}$$

$$p_{2r_{\odot}} = [(0.06*0.2) + (0.18*0.7) + (0.15*0)] * 0.2 = \underline{0.0276}$$

t=3:

$$p_{3r_{\otimes}} = [(0.0798*0) + (0.0276*0.3) + (0.0276*0.4)] * 0 = \underline{0}$$

$$p_{3r_{\odot}} = [(0.0798*0.8) + (0.0276*0) + (0.0276*0.6)] * 0.2 = \underline{0.01608}$$

$$p_{3r_{\odot}} = [(0.0798*0.2) + (0.0276*0.7) + (0.0276*0)] * 0.5 = \underline{0.01764}$$

Frage: wie groß ist die WK dass die Notenkette {2,3,1} vergeben wird ?

Terminierung: $P(D|\theta) = \sum_r p_{\text{Trs}}$

$$P(D|\theta) = 0 + 0.01608 + 0.01764 = \underline{\underline{0.03372}}$$

Aufsummieren der WK's

Matlab: Prof.m

Analog kann man auch *rückwärts* gehen. Initialisierung erfolgt mit

1. Initialisierung $\beta_T(i) = 1, i=1, \dots, N$ (Formal für die Rekursion)

2. Rekursion,
$$\beta_t(i) = \sum_{j=1}^N a_{ij} b_j(o(t+1)) \beta_{t+1}(j)$$
$$i=1, \dots, N, t = T-1, T-2, \dots, 1$$

3. Terminierung $P(o(1)o(2)\dots o(T)) = \sum_{j=1}^N p_j b_j(o(1)) \beta_1(j)$

Das Ergebnis vom Forward-Algorithmus ist gleich dem Backward-Algorithmus (Matlab Example)

2. Grundlegende Fragestellung

Gegeben ist ein HMM λ und die Beobachtung O . Gesucht ist Wahrscheinlichkeit $P(z_t = i \mid O)$. Also die Wahrscheinlichkeit, dass sich das HMM zur Zeit t im Zustand i befand, unter der Bedingung dass die Ausgabe O beobachtet wurde. Mit Hilfe des Forward-Backward-Algorithmus kann diese Wahrscheinlichkeit effizient berechnet werden.

Gesucht: $\lambda_t(i) = P(q(t) = q_i \mid O)$

Lösung:

$$\lambda_t(i) = \alpha_t(i) \beta_t(i) / P(O), i = 1, \dots, N, t = 1, \dots, T$$

3. Grundlegende Fragestellung

Gegeben ist ein HMM λ . Es soll die wahrscheinlichste Sequenz der (versteckten, hidden) Zustände bestimmt werden, die eine vorgegebene Ausgabesequenz erzeugt hat. Lösbar mit Hilfe des Viterbi-Algorithmus.

Der **Viterbi-Algorithmus** ist ein Algorithmus der Dynamische Programmierung zur Bestimmung der wahrscheinlichsten Sequenz von versteckten Zuständen bei einem gegebenen Hidden Markov Model und einer beobachteten Sequenz von Symbolen. Diese Zustandssequenz wird auch als **Viterbi-Pfad** bezeichnet.

$$\delta_t(i) = \max\{P(q(1), q(2), \dots, q(t-1) ; o(1), o(2), \dots, o(t) \mid q(t) = q_i).\}$$

<http://de.wikipedia.org/wiki/Viterbi-Algorithmus>

3. Grundlegende Fragestellung

Das gleiche Schema wie im Forward-Algorithmus, nur dass die Summe durch den maximal-Wert ersetzt wird und gespeichert wird von welchem Zustand aus die WK maximiert wird (für Backtracking).

1. Initialisierung $\delta_1(i) = p_i b_i(o(1))$
 $\psi_1(i) = 0, i = 1, \dots, N$
2. Rekursion, $\delta_t(j) = \max_i [\delta_{t-1}(i) a_{ij}] b_j(o(t))$
 $\psi_t(j) = \arg \max_i [\delta_{t-1}(i) a_{ij}]$
3. Terminierung $p^* = \max_i [\delta_T(i)]$
 $q^*_T = \arg \max_i [\delta_T(i)]$

Backtracking:

$$q^*_t = \psi_{t+1}(q^*_{t+1}), t = T-1, T-2, \dots, 1$$

Der Professor hat jeden Tag einen Studenten benotet.
Zu der gegebenen Beobachtung {2, 3, 1, 2, 1} sollen nun die
wahrscheinlichsten Gemütszustände bestimmt werden:

Anfangsw-keiten:

$$q(r, \text{☹}) = 0.2$$

$$q(r, \text{☺}) = 0.3$$

$$q(r, \text{😊}) = 0.5$$

Noten Gemüt	1	2	3
☹	0.0	0.3	0.7
☺	0.2	0.6	0.2
😊	0.5	0.3	0.2

Bei der Initialisierung werden die W.-keiten für den Anfangs-Gemütszustand mit der jeweiligen Ausgabewahrscheinlichkeit für die Note 2 multipliziert.

Initialisierung:

$$L_1(s) = \pi_0(s) * P_s(d_1|\theta)$$

$$L_1(\text{☹}) = 0.2 * 0.3 = 0.06$$

$$L_1(\text{☺}) = 0.3 * 0.6 = 0.18$$

$$L_1(\text{😊}) = 0.5 * 0.3 = 0.15$$

Rekursion:

$$L_t(s) = \max_{a_{rs} = L_{t-1}(r) * q(r,s)} [L_{t-1}(r) * q(r,s)] * P_s(d_t|\theta)$$

			
	0	0.8	0.2
	0.3	0	0.7
	0.4	0.6	0

3. Grundlegende Fragestellung

Zeit	1	2	3	4	5
Zu-stand	2	3	1	2	1
	0.06	$a_{\text{sad sad}} = 0.18 * 0.3 = 0.054$ $a_{\text{happy sad}} = 0.15 * 0.4 = \underline{0.06}$ $L_2 = 0.7 * 0.06 = \mathbf{0.042}$	$L_t = \mathbf{0}$	$L_t = \mathbf{0.000756}$	$L_t^{\sim} = \mathbf{0}$
	0.18	$a_{\text{sad neutral}} = 0.06 * 0.8 = 0.048$ $a_{\text{happy neutral}} = 0.15 * 0.6 = \underline{0.09}$ $L_2 = 0.2 * 0.09 = \mathbf{0.018}$	$L_t = \mathbf{0.00672}$	$L_t = \mathbf{0.002268}$	$L_t^{\sim} = \mathbf{0.00016}$
	<u>0.15</u>	$a_{\text{sad happy}} = 0.06 * 0.2 = 0.012$ $a_{\text{happy happy}} = 0.18 * 0.7 = \underline{0.126}$ $L_2 = 0.2 * 0.126 = \mathbf{0.025}$	$L_t = \mathbf{0.0063}$	$L_t = \mathbf{0.001411}$	$L_t^{\sim} = \mathbf{0.0015}$

ProfVitebri.m

Tippfehler!

3. Grundlegende Fragestellung

Bei der Terminierung wird ausgehend von der maximalen Wahrscheinlichkeit für die Vergabe der letzten Note, die Sequenz der Gemütszustände rückwärts ausgewertet.

$$H' = \{ h_1, h_2, \dots, h_5 \} = \{ \text{😊} , \text{😐} , \text{😊} , \text{😐} , \text{😊} \}$$

4. Lernproblem

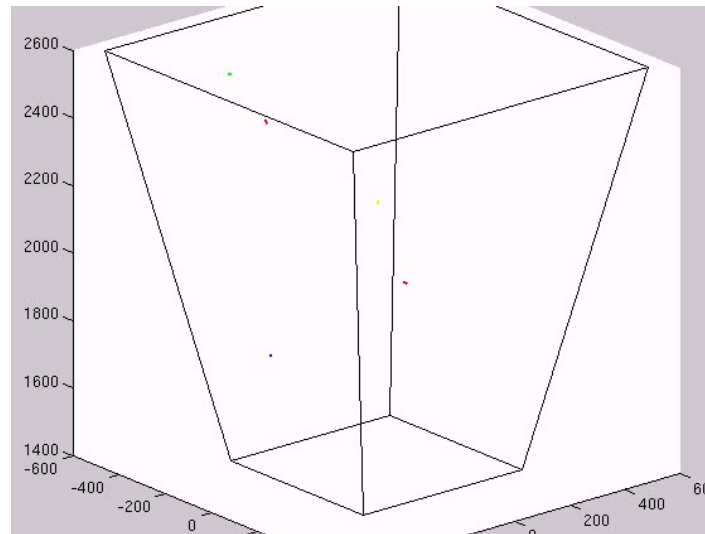
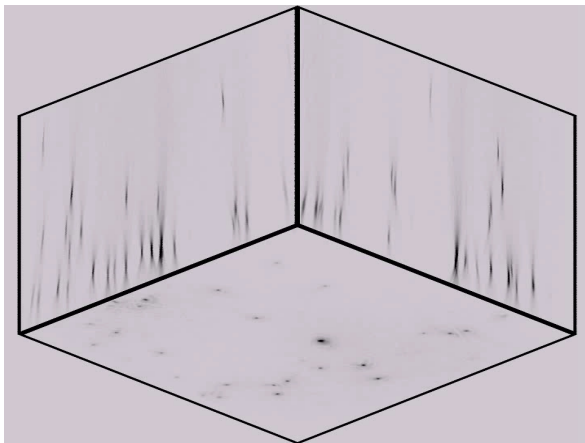
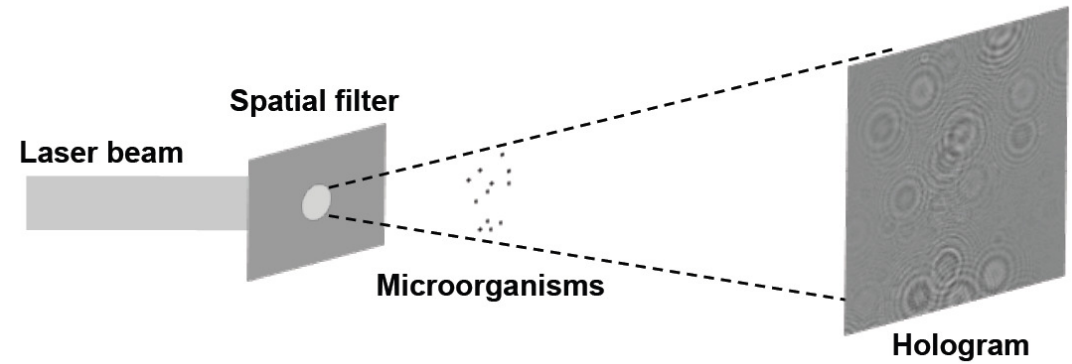
Gegeben ist nur die Ausgabesequenz. Es sollen die Parameter des HMM bestimmt werden, die am wahrscheinlichsten die Ausgabesequenz erzeugen. Lösbar mit Hilfe des Baum-Welch-Algorithmus.

Der Baum-Welch-Algorithmus ist ein erwartungsmaximierender Algorithmus. Er berechnet die Maximalwahrscheinlichkeitsschätzungen (Maximum-Likelihood-Schätzwerte) und die sogenannten *Posterior-Mode-Schätzungen* für die gegebenen Parameter (Übergangs- und Emissionswahrscheinlichkeit) eines HMMs, wenn nur die Emissionen als Trainingsdaten gegeben sind.

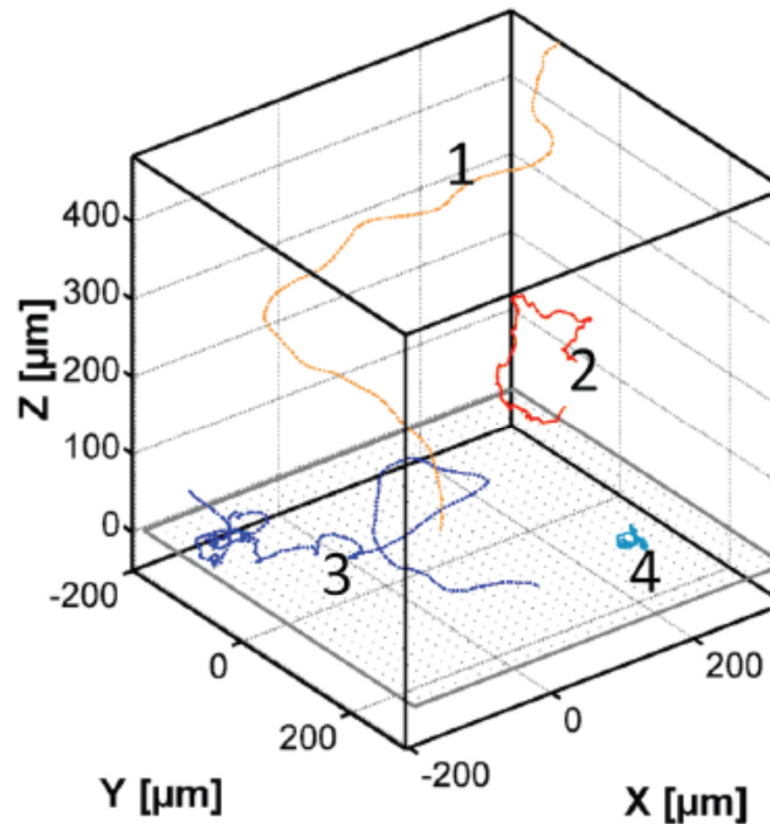
(nicht prüfungsrelevant)

<http://de.wikipedia.org/wiki/Baum-Welch-Algorithmus>

Digitale Inline-Holographie



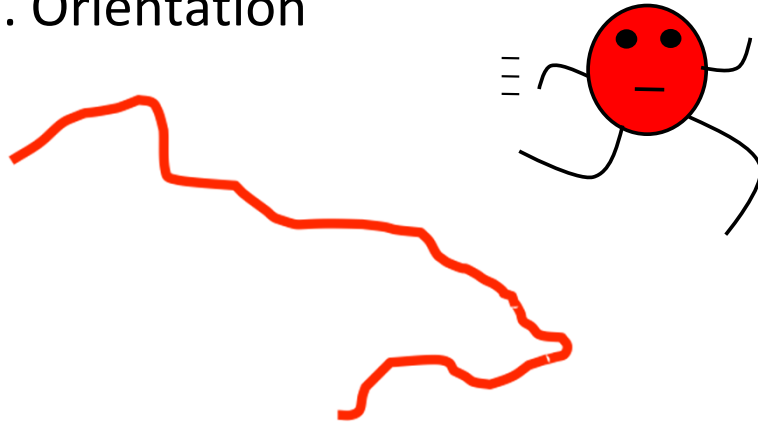
DFG: RO 2497/ 5, 6
Leal et.al. IEEE WMVC09



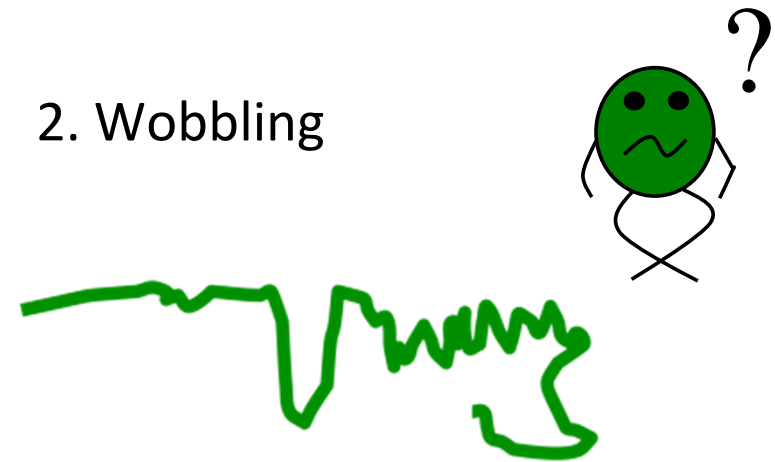
Dynamische Analyse von Schwimm-Mustern (Mikroorganismen)

Analysis of motion patterns

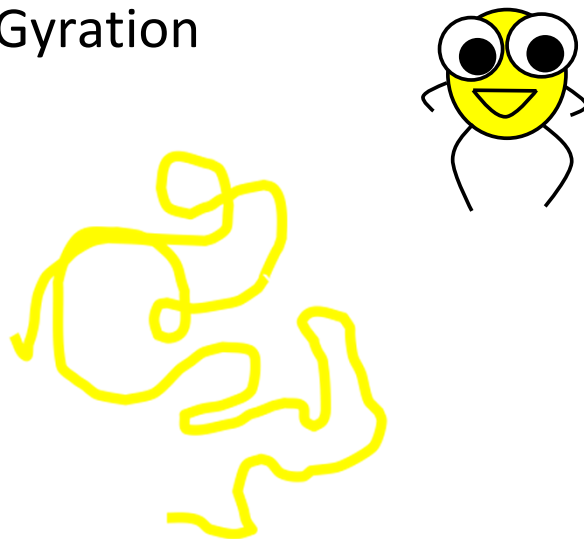
1. Orientation



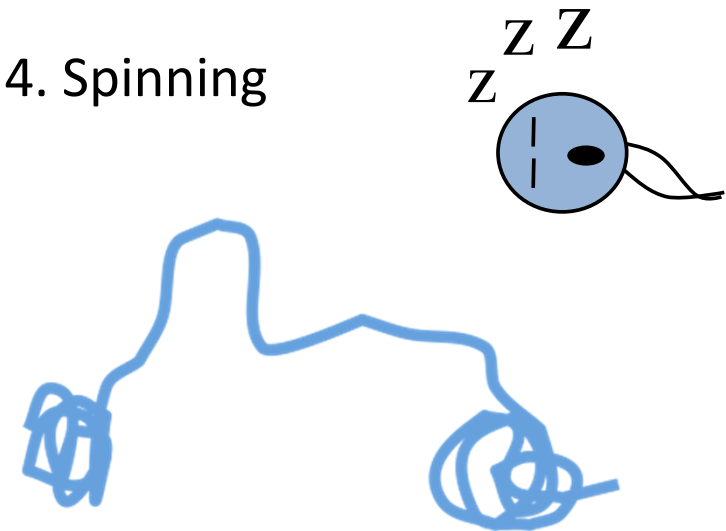
2. Wobbling

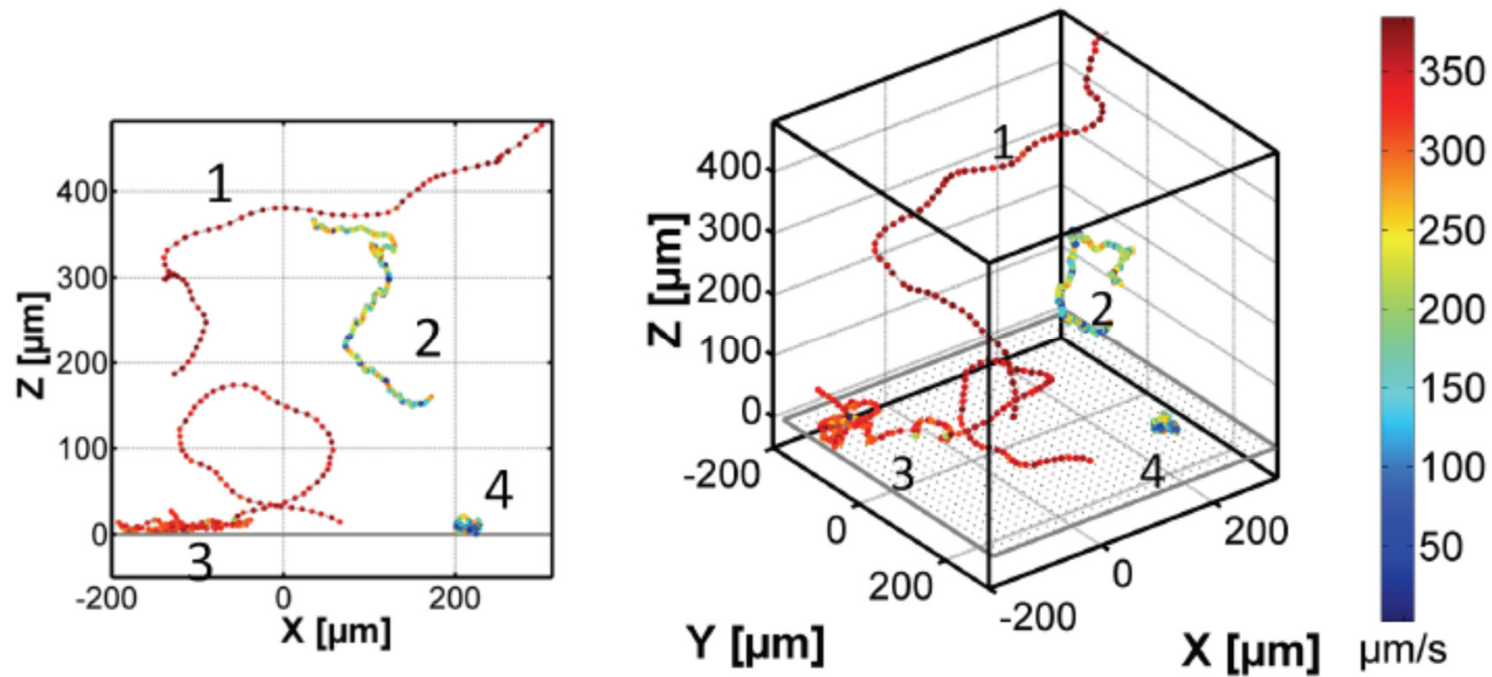


3. Gyration

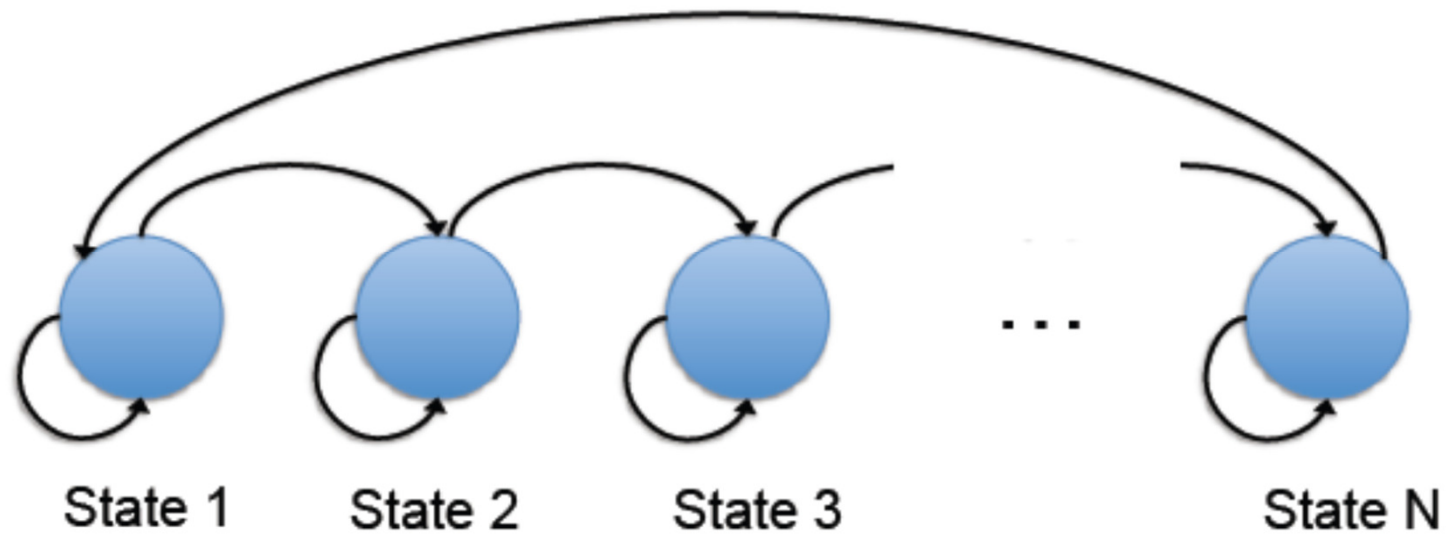


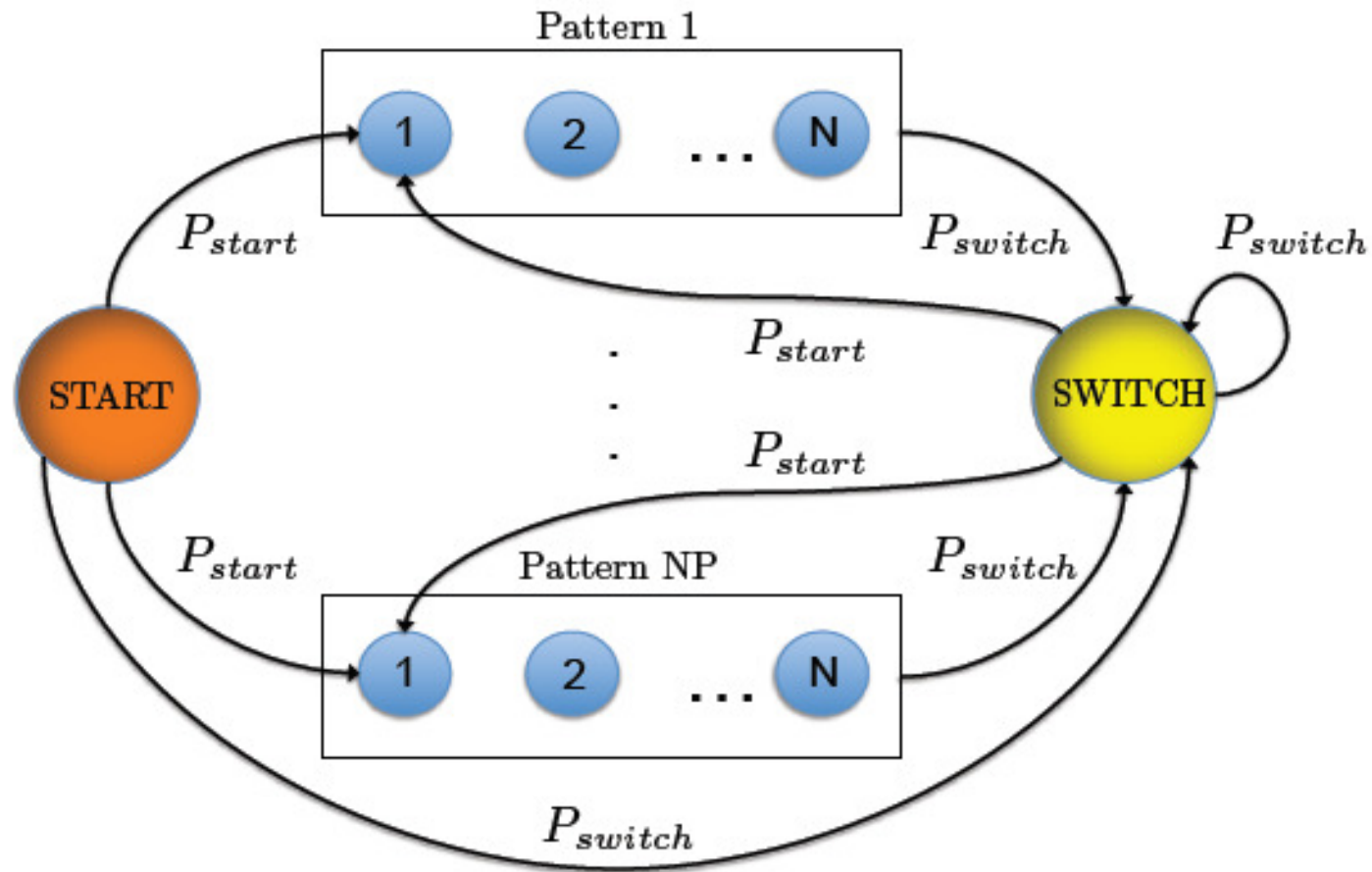
4. Spinning



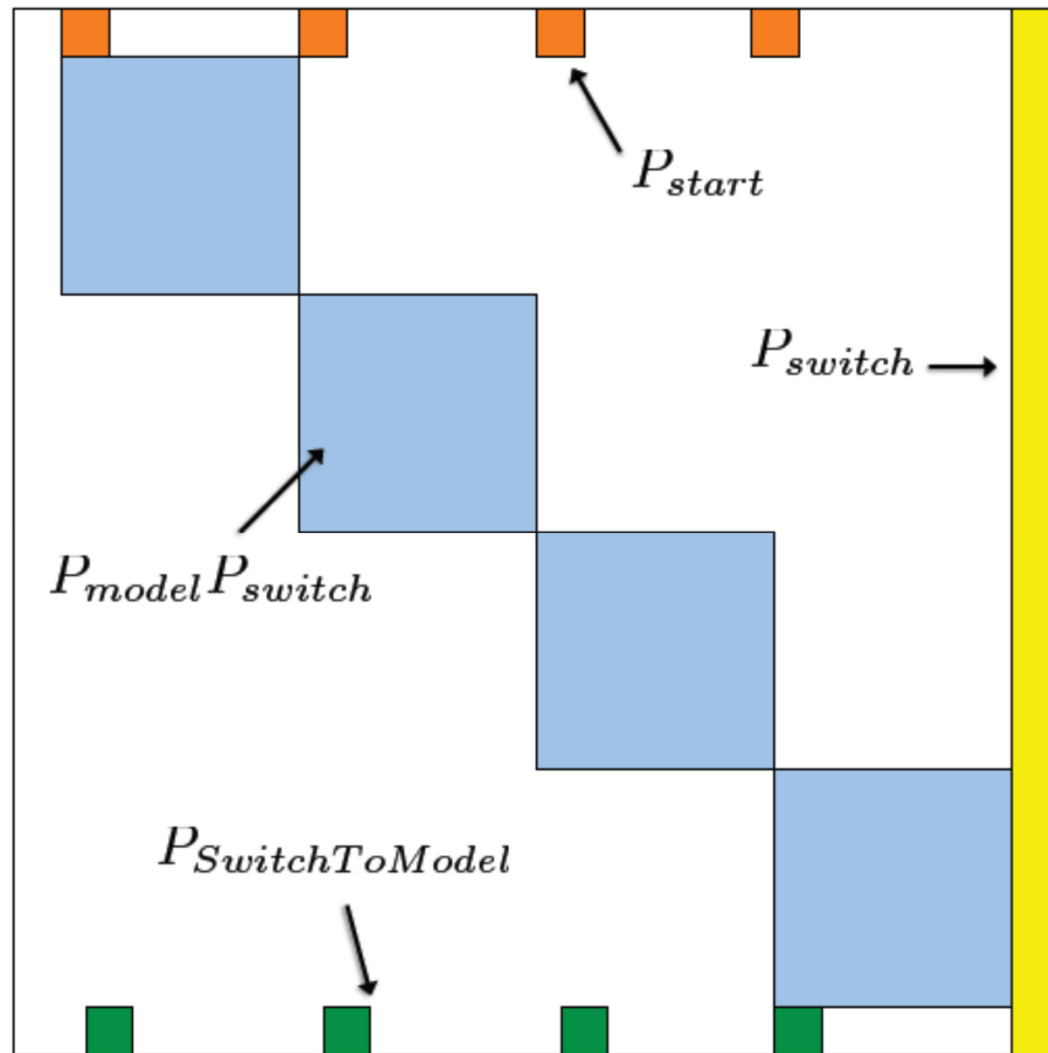


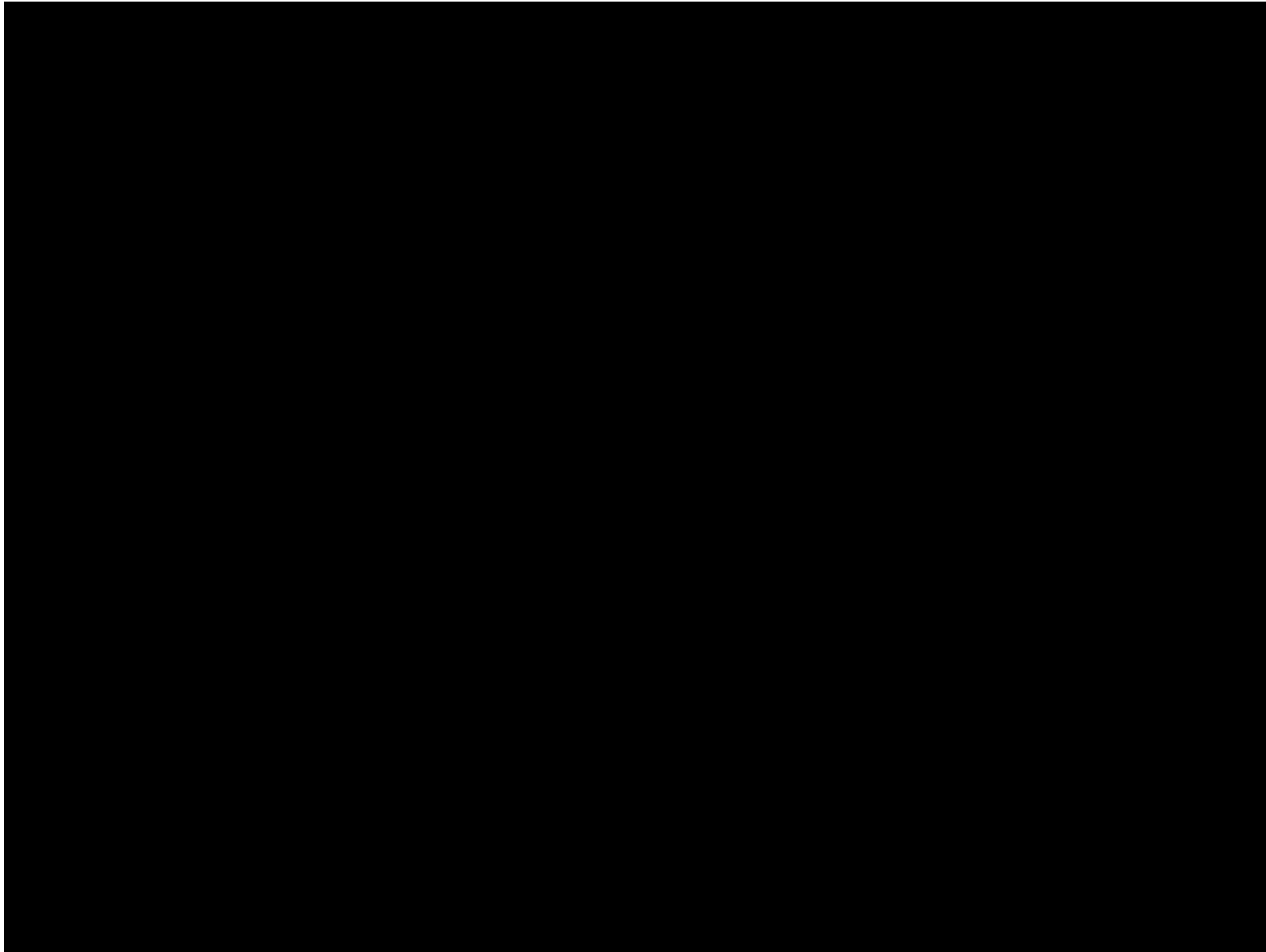
Drift, Wobbling, Gyration, Spinning





Hidden Markov Modelle

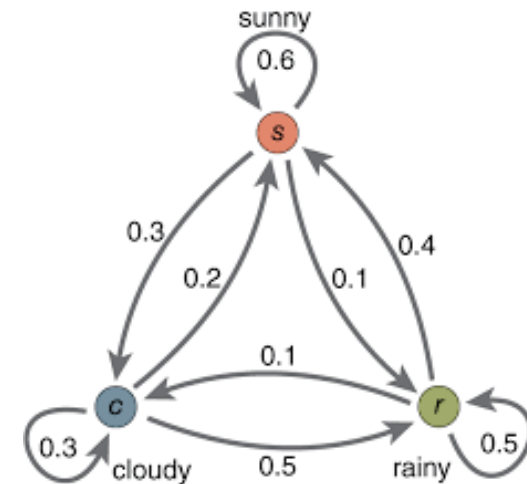
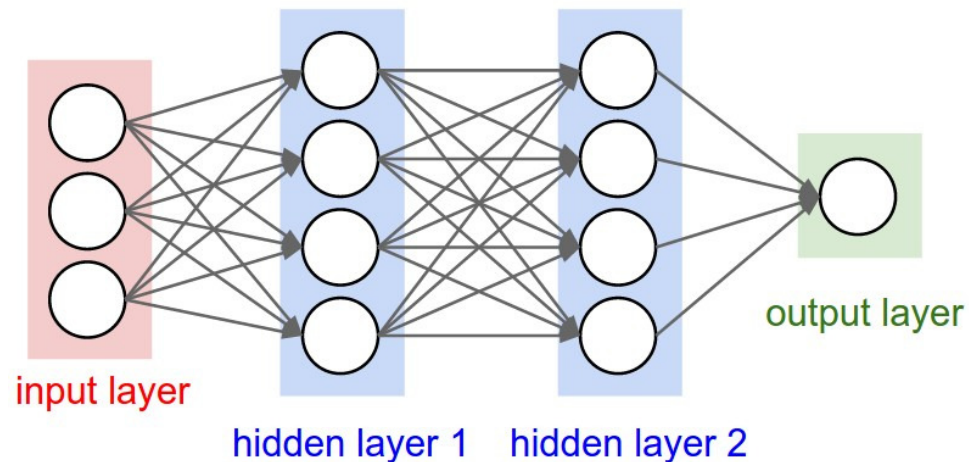




NNs vs HMMs

Development of a Neural network (NN) which is capable to simulate Markov Chains.

Awiszus et.al. CVPR-WS 2018



Neural Networks:

- Generalized approximator for any function
- Deterministic
- State-of-the Art

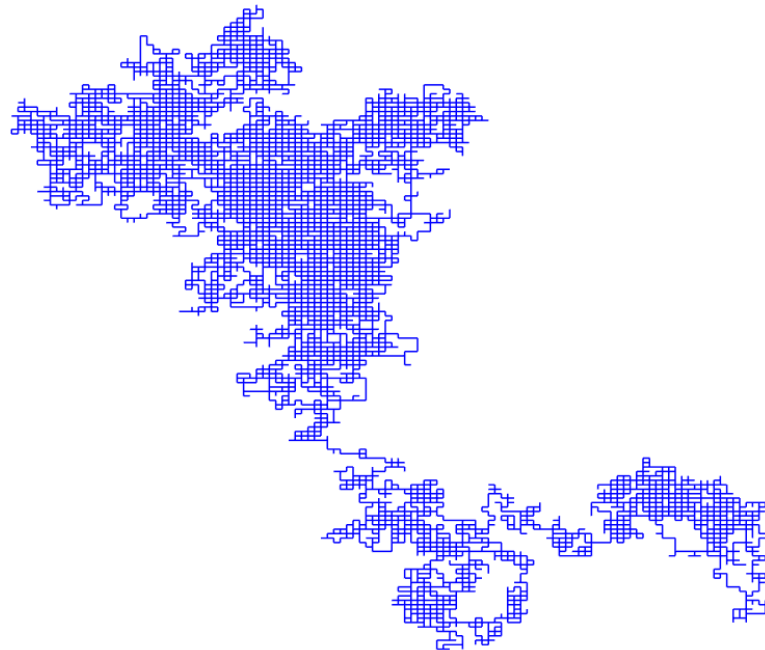
Markov Chain:

- Discrete state space
- Transition probabilities
- Stochastic process

However ...

Many processes are non-deterministic :

- Chat Bots for HCI
- AIs in Games
- Random Walker (Fluid Dynamics, Diffusion models, Brownian motions, etc.)

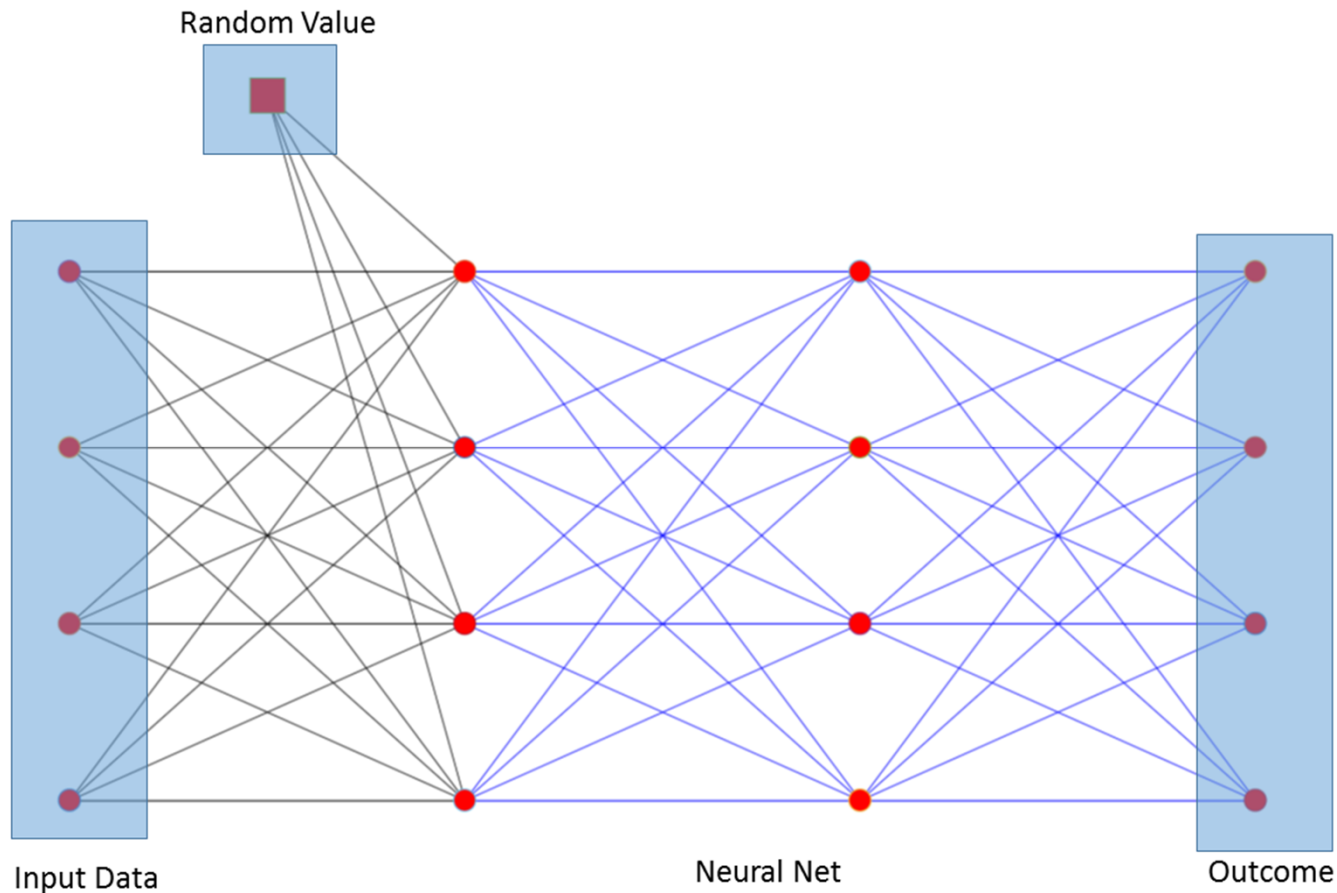


Goal: How to teach a Neural Network non-deterministic behavior ?

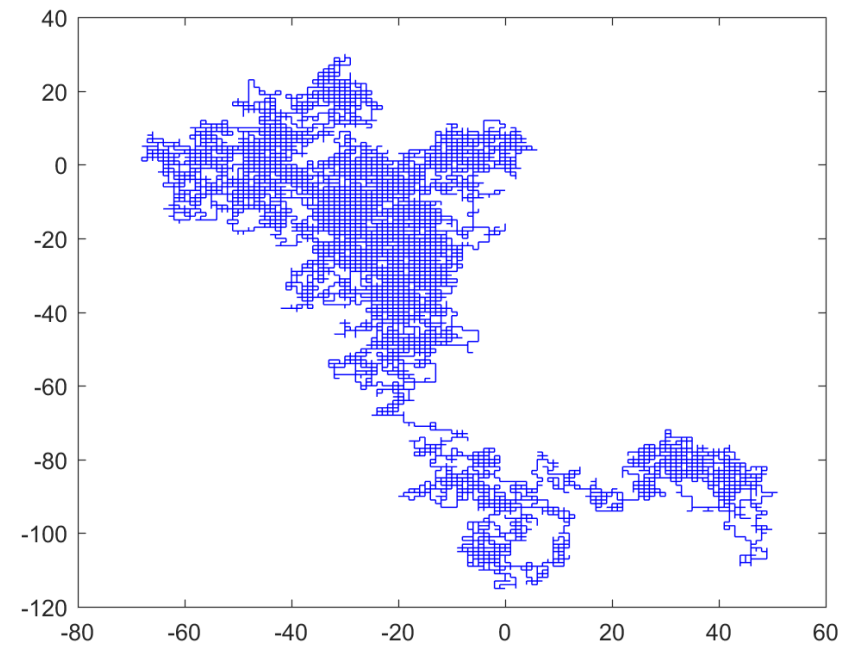
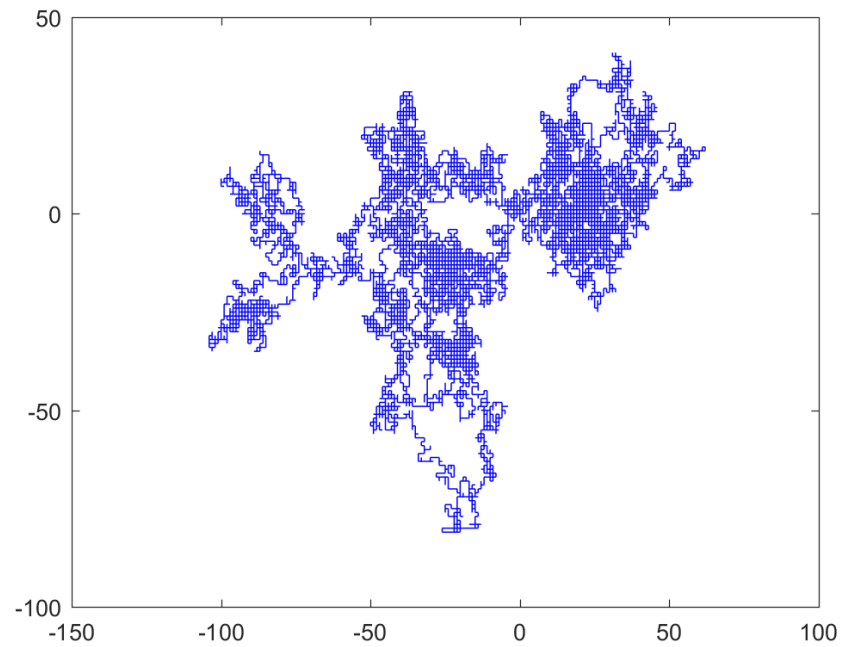
Markov Chain Neural Networks

Goal: How to teach a Neural Network non-deterministic behavior ?

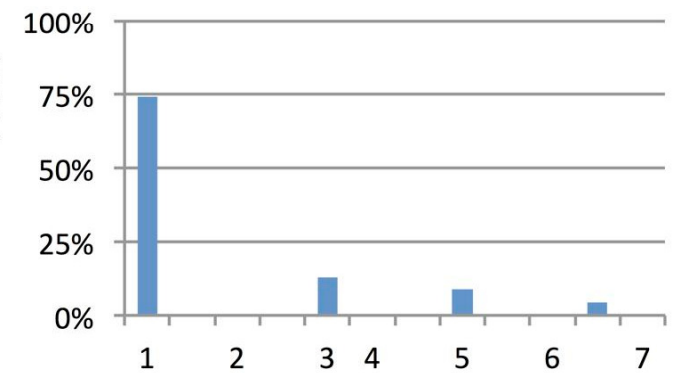
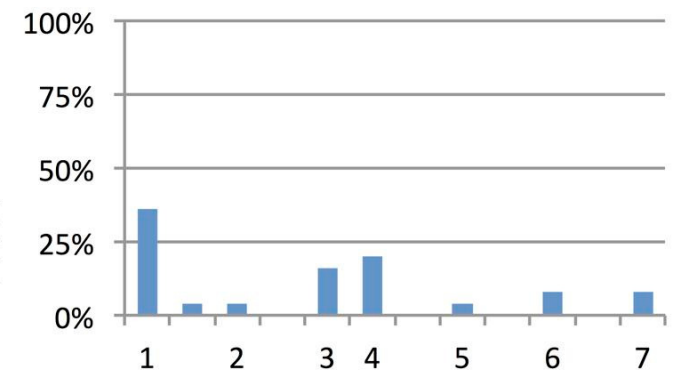
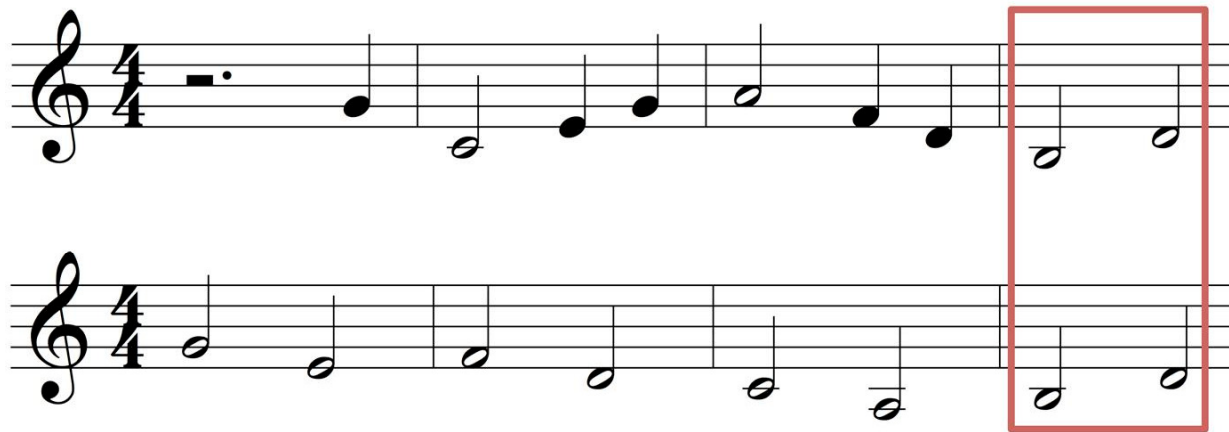
Idea: Add an additional input variable which contains a random value.
This random value will act as a *switch* to ensure different outcomes



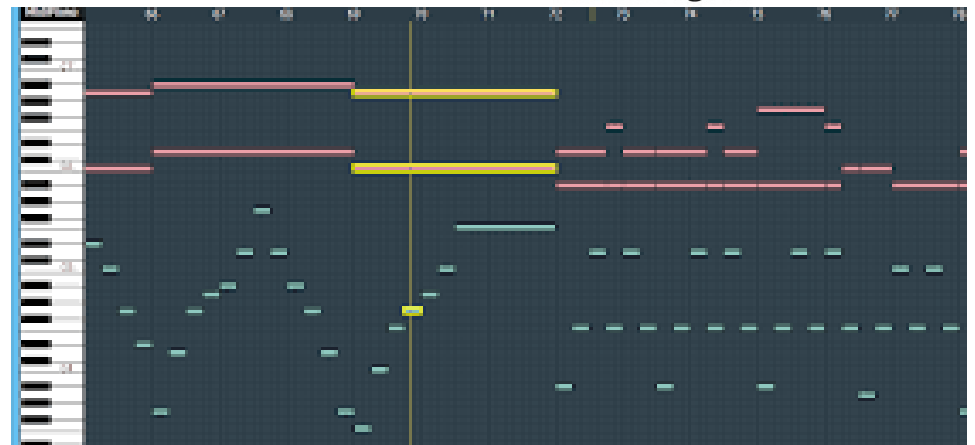
Random Walker Net



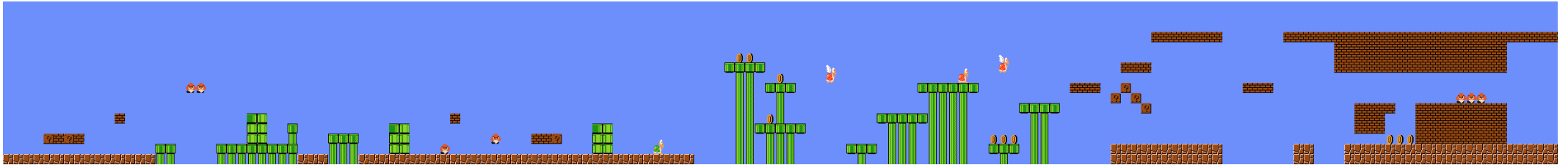
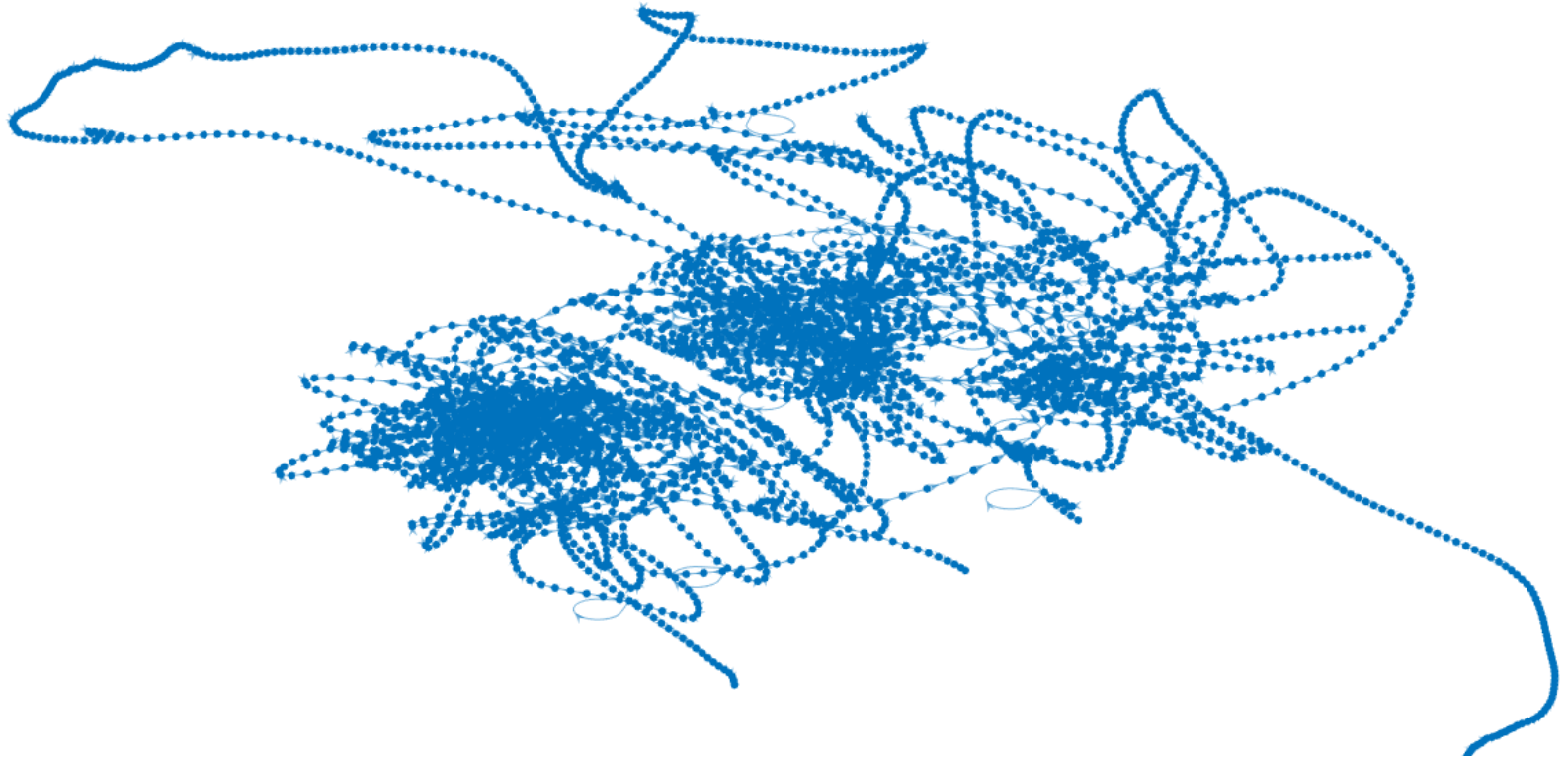
Predictive Music



Scale Degree



Mario Level Design



- VMM
- HMM
 - WK einer Beobachtung
 - WK eines Zustandes gegeben eine Beobachtung
 - größte WK verborgender Zustände gegeben Beobachtung
 - Anpassung Modellparameter gegeben Beobachtungen
- NNs 4 HMMs